

CS 423

Operating System Design:

More Page Tables

Sep 22

Ram Alagappan

Slide ack: Prof. Shivaram Venkataraman (Wisconsin)

Logistics

MP0 grades released (over email)

MP1 due 09/30 - Utilize TA office hours if you need help

AGENDA / LEARNING OUTCOMES

Recap TLBs

Smaller page tables: tackle memory overheads of page tables

RECAP

Disadvantages of Paging

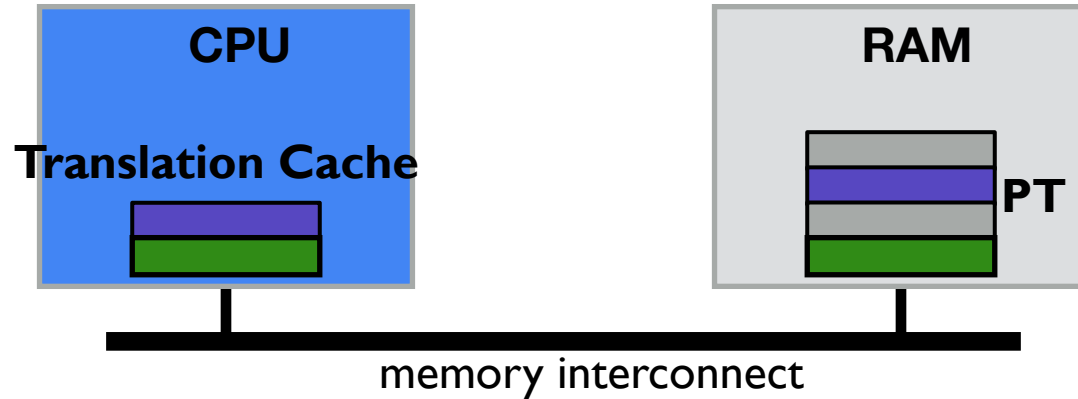
Additional memory reference to page table entry

- Extra memory access needed for each memory access!
- Inefficient, even if page table is stored in memory
- MMU stores only base address of page table
 - Solution: TLBs

Space needed for page tables is too large

- Simple page table: requires PTE for each virtual page number
- PTE needed even if that page is never allocated
- Page tables must be contiguously allocated
 - Solution: paging the page tables!

TLB: CACHE PAGE TRANSLATIONS



TLB: TRANSLATION LOOKASIDE BUFFER

PAGE TRANSLATION WITH TLB

For each mem reference:

1. extract **VPN** (virt page num) from **VA** (virt addr)
2. check TLB for **VPN**
 if miss:
 3. calculate addr of **PTE** (page table entry)
 4. read **PTE** from memory, insert into TLB, retry instruction
5. extract **PFN** from TLB (page frame num)
6. build **PA** (phys addr)
7. read contents of **PA** from memory

HW AND OS ROLES

Who handles TLB hit?

Who handles TLB miss?

TLB MISS - HW AND OS ROLES

If HW, then HW must know where the PT is in memory

CR3 in x86; page table structure agreed upon between OS and HW

Hardware “walks” page table structure, fills in TLB

If OS (“software managed TLB”)

HW traps into OS (kernel mode) upon TLB miss

OS walks page table structure (designed by OS writers), fills in TLB

Returns to user-mode

Retry the same instruction in the user-process => TLB hit

TLB Summary

Paging is great, but accessing page tables for every memory access is slow

Cache recent page translations in TLB

- MMU performs TLB lookup on every memory access

TLB performance depends strongly on workload

- Sequential workloads perform well
- Workloads with temporal locality can perform well

Upon context switches

- Flush TLB on every context switch
- Add ASID to each TLB entry

TLB miss handling: hardware or OS

Trade-off: speed vs flexibility

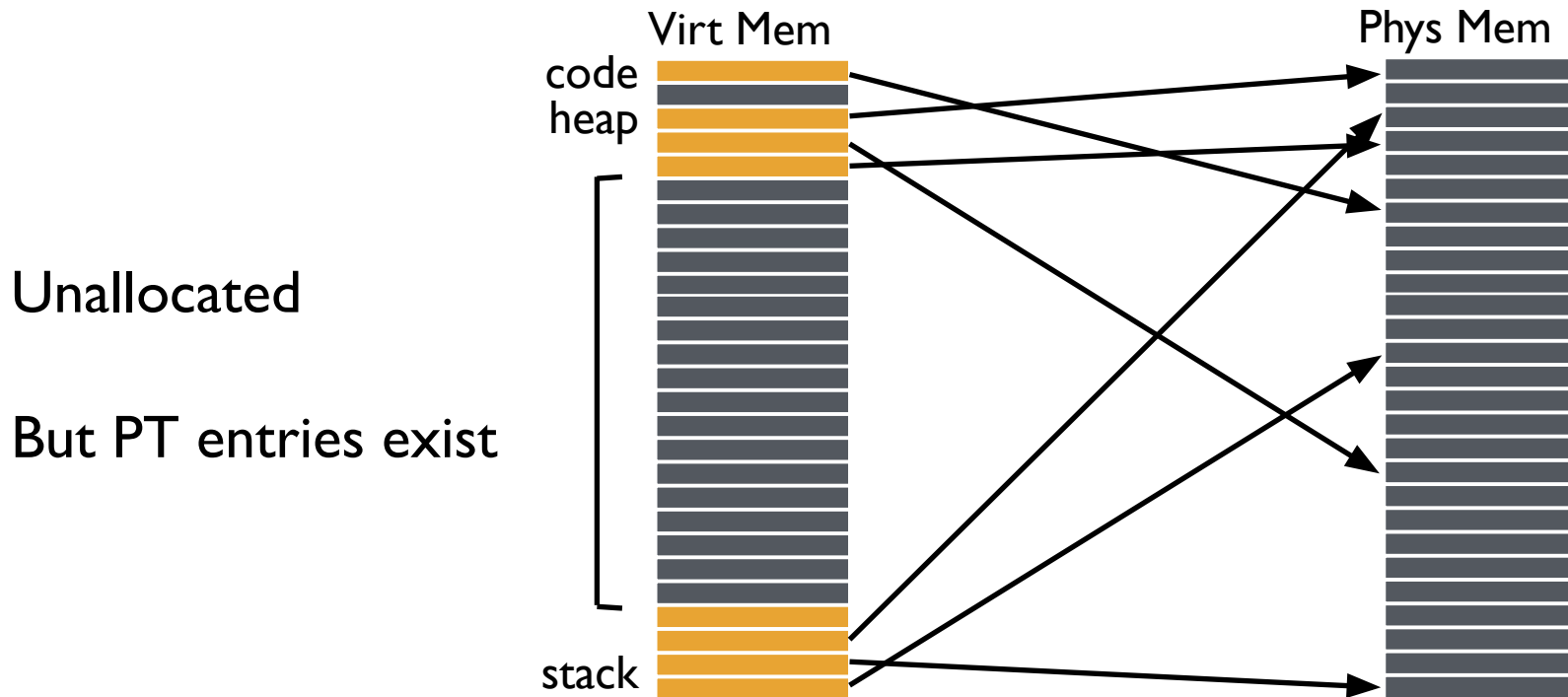
END RECAP

Today's class: space needed for PT

Space for entire page tables can be substantial

- Simple page table: Require PTE for all pages in address space
- Better ways to save space...

Why are Page Tables so Large?



MANY INVALID PTES

PFN	valid	prot
10	1	r-x
-	0	-
23	1	rw-
-	0	-
-	0	-
-	0	-
-	0	-
...many more invalid...		
-	0	-
-	0	-
-	0	-
-	0	-
28	1	rw-
4	1	rw-

how to avoid
storing these?

Problem: linear PT must still allocate PTE for
each page (even unallocated ones)

DON'T NEED A FLAT PAGE TABLE

Page table is just another data structure

Use more complex page tables, instead of just big array

Any data structure (e.g., hash table) is possible if OS handles TLB miss

- MMU looks up vpn in TLB on every memory access
- On TLB miss
 - Trap into OS and let OS find vpn -> ppn translation
 - OS inserts PTE into TLB

In principle, you could also have the hardware do the same... but...

VARIOUS APPROACHES

1. Segmented paging
2. Multi-level page tables
 - Page the page tables
 - Page the page tables of page tables...
3. Inverted page tables

VALID PTES ARE CONTIGUOUS

how to avoid
storing these?

PFN	valid	prot
10	1	r-x
-	0	-
23	1	rw-
-	0	-
-	0	-
-	0	-
-	0	-
...many more invalid...		
-	0	-
-	0	-
-	0	-
-	0	-
28	1	rw-
4	1	rw-

Note “hole” in addr space:
valids vs. invalids are clustered

How to avoid allocating holes in the page
table?

Combination: Segmented Paging

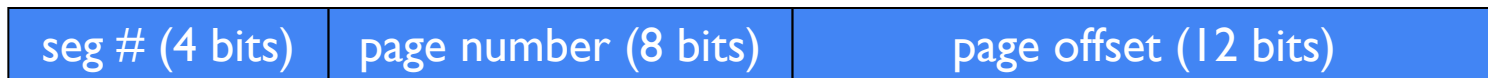
Used by early systems, esp. Multics in 1960s, not today's Linux or Windows

Divide address space into segments (code, heap, stack)

- Segments can be variable length

Divide each segment into fixed-sized pages

Logical address divided into three portions



- Each segment has a page table
- Track base physical address and bounds of the **page table** per segment
 - This is different from original segmentation
 - Bounds => # PTEs in segment

Segmented Paging

seg # (4 bits)	page number (8 bits)	page offset (12 bits)
----------------	----------------------	-----------------------

seg	base	bounds	R W
0	0x002000	0xff	1 0
1	0x000000	0x00	0 0
2	0x001000	0x0f	1 1

Bounds = #PTEs

Why does this save space?

Don't need mapping for unallocated region between the heap and the stack

Base: physical addr of PT (PT is linear!)

Bounds: number of entries in the PT

...	
0x01f	0x001000
0x011	
0x003	
0x02a	
0x013	
...	
0x00c	0x002000
0x007	
0x004	
0x00b	
0x006	
...	

Segmented Paging

seg # (4 bits)	page number (8 bits)	page offset (12 bits)
----------------	----------------------	-----------------------

seg	base	bounds	R W
0	0x002000	0xff	1 0
1	0x000000	0x00	0 0
2	0x001000	0x0f	1 1

Bounds = #PTEs

0x002070 read: ???

0x202016 read: ???

0x104c84 read: ???

0x010424 write: ???

0x210014 write: ???

...	
0x01f	0x001000
0x011	
0x003	
0x02a	
0x013	
...	
0x00c	0x002000
0x007	
0x004	
0x00b	
0x006	
...	

Segmented Paging

seg # (4 bits)	page number (8 bits)	page offset (12 bits)
----------------	----------------------	-----------------------

seg	base	bounds	R W
0	0x002000	0xff	1 0
1	0x000000	0x00	0 0
2	0x001000	0x0f	1 1

Bounds = #PTEs

0x002070 read: 0x004070

0x202016 read: 0x003016

0x104c84 read: Error (bounds or rw perm)

0x010424 write: Error (read-only perm)

0x210014 write: Error (bounds 0x10 > 0x0f)

...	
0x01f	0x001000
0x011	
0x003	
0x02a	
0x013	
...	
0x00c	0x002000
0x007	
0x004	
0x00b	
0x006	
...	

Questions

True or False:

1. With segmented paging, a segment can grow by X MBs even when there's no contiguous X MBs of physical memory available
2. The above is not possible in pure segmentation
3. If heap has 100 pages, then there are 100 PTEs in that segment's PT
4. If the first 99 pages are freed in the heap, the PT now has one PTE

Advantages & Disadvantages

Advantages:

- Reduces external fragmentation (unlike Segmentation)
- Segments can grow without the need to reshuffle

Disadvantages:

- Must allocate page table for each segment *contiguously* (still a linear PT!)
- Page table size?
 - Assume 2 bits for segment, 18 bits for page number, 12 bits for offset

Each page table can grow to:

= Number of entries * size of each entry

= Number of pages * 4 bytes

= $2^{18} * 4 \text{ bytes} = 2^{20} \text{ bytes} = 1 \text{ MB!}$

Sparse address spaces still take up a lot of space for PTEs

OTHER APPROACHES

1. Segmented paging
2. Multi-level page tables
 - Page the page tables
 - Page the page tables of page tables...
3. Inverted page tables

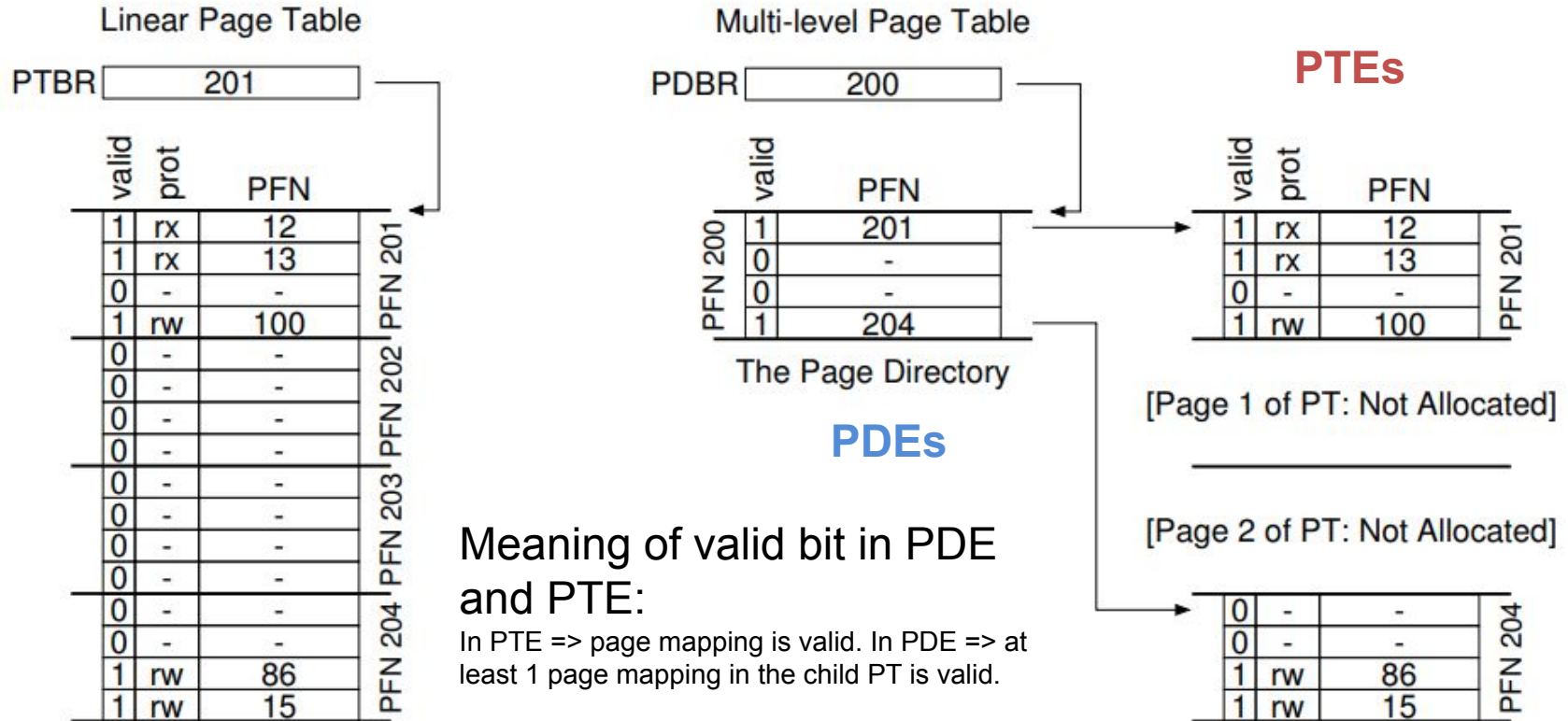
Multi-level Page Tables

Goal: Allow page table to be allocated non-contiguously

Idea: Page the page tables!

- Creates multiple levels of page tables; outer level “page directory”
- Only allocate page tables for pages in use → saves space even with sparse address space

Multilevel Page Table – Key Idea



Questions

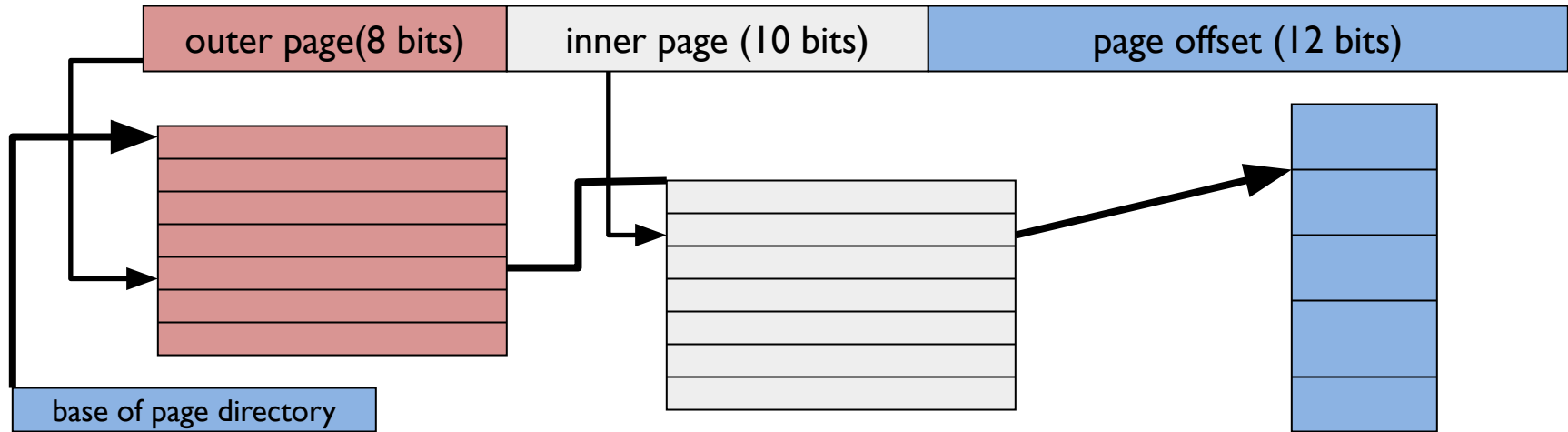
Can pages of the PT be dynamically relocated to a different physical location?
How?

Can the PD be dynamically relocated to a different physical location? How?

How many memory accesses on a TLB miss with this 2-level PT?

Multilevel Page Tables

30-bit address:



Address format for multi-level paging

30-bit address:



How should logical address be structured? How many bits for each paging level?

Page size dictates #bits for offset

4KB page => 12 bit offset

Goal: each inner page table fits within a page

PTE size * number PTEs = page size

Assume PTE size = 4 bytes

Each inner page can have 1024 PTEs

=> 10 bits for inner page

Remaining bits for outer page:

=> $30 - 12 - 10 = 8$ bits

Multilevel Translation EXAMPLE

page directory

PPN	valid
0x3	1
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
0x92	1

page of PT (@PPN:0x3)

PPN	valid
0x10	1
0x23	1
-	0
-	0
0x80	1
0x59	1
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0

page of PT (@PPN:0x92)

PPN	valid
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
0x55	1
0x45	1

translate 0x01ABC

Physical address?

Look PD[0]

Look PT[1] → arrive at

page 0x23

Concat ABC to arrive at

0x23ABC

outer page(4 bits)

inner page(4 bits)

page offset (12 bits)

20-bit
address:

Multilevel Translation EXAMPLE

page directory

PPN	valid
0x3	1
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
0x92	1

page of PT (@PPN:0x3)

PPN	valid
0x10	1
0x23	1
-	0
-	0
0x80	1
0x59	1
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0

page of PT (@PPN:0x92)

PPN	valid
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
0x55	1
0x45	1

Translate

VA: 0x04000

PD[0]->PT[4] = 0x80

PA = 0x80000

VA: 0xFEED0

PD[f]->PT[e] = 0x55

PA = 0x55ED0

outer page(4 bits)

inner page(4 bits)

page offset (12 bits)

20-bit
address:³¹

PROBLEM WITH 2 LEVELS?

Problem: page directories (outer level) may not fit in a page

Consider 30 bit address with 512-byte pages

=> 9 bits for offset; leaves 21 bits for VPN

Remember our goal: each inner page's PTEs should fit within a page

So how many PTEs per page? With 512-byte pages and 4-byte PTE, we can have 128 entries → this means 7 bits for inner page, leaving 14 bits for outer page (or directory) → 2^{14} PDEs

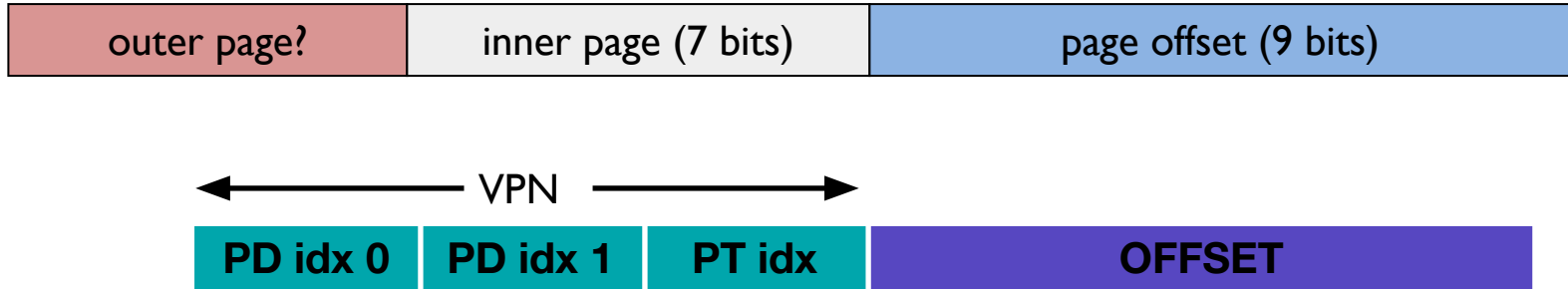
Assume 4-byte PDE, then PD itself will span _____ pages?

PD cannot be contained in one page now!

PROBLEM WITH 2 LEVELS?

Solution: page the page directory!

Add another level of page directory that points to PD pages



Can keep going recursively!

#Memory Accesses Exercise

Assume 3-level page table

Assume 256-byte pages

Assume 16-bit addresses

Assume ASID of current process is 211

How many physical accesses for each instruction?

(a) 0xAA10: movl 0x1111, %edi

(b) 0xBB13: addl \$0x3, %edi

(c) 0x0519: movl %edi, 0xFF10

ASID	VPN	PFN	Valid
211	0xbb	0x91	1
211	0xff	0x23	1
122	0x05	0x91	1
211	0x05	0x12	0

(Ignore ops changing TLB)

FULL SYSTEM WITH TLBS

Assume 3-level page table

Assume 256-byte pages

Assume 16-bit addresses

Assume ASID of current process is 211

ASID	VPN	PFN	Valid
211	0xbb	0x91	1
211	0xff	0x23	1
122	0x05	0x91	1
211	0x05	0x12	0

How many physical accesses for each instruction?

(Ignore ops changing TLB)

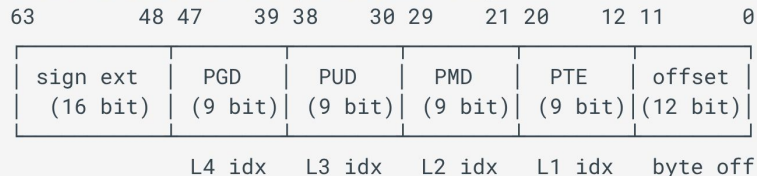
1. 0xaa: (TLB miss -> 3 for addr trans) + 1 instr fetch
0x11: (TLB miss -> 3 for addr trans) + 1 movl
2. 0xbb: (TLB hit -> 0 for addr trans) + 1 instr fetch from 0x9113
3. 0x05: (TLB miss -> 3 for addr trans) + 1 instr fetch
0xff: (TLB hit -> 0 for addr trans) + 1 movl into 0x2310

TLB caches translations for the page table themselves

On TLB miss: lookups with more levels more expensive

Linux x86 Page Table

48-bit virtual address (canonical form):



Terminology used in the kernel source:

PGD = Page Global Directory (level 4, pointed to by CR3)

PUD = Page Upper Directory (level 3)

PMD = Page Middle Directory (level 2)

PTE = Page Table Entry (level 1, maps a 4KB page)

CR3 (physical addr of PGD)

→ PGD[VA[47:39]] (physical addr of PUD page)

→ PUD[VA[38:30]] (physical addr of PMD page)

→ PMD[VA[29:21]] (physical addr of PTE page)

→ PTE[VA[20:12]] (physical addr of 4KB page frame)

+ VA[11:0] (byte offset within the page)

= Physical Address

Remember: TLB caches translations for these tables as well (not just last level)

5-level paging: modern x86 can extend 48-bit VA to 57-bit VA by adding another level.

Huge Pages in x86 Linux

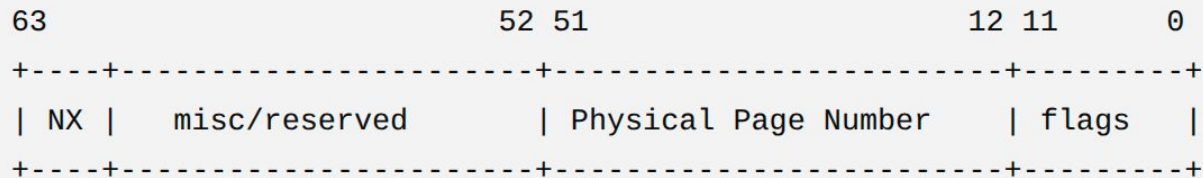
VA
[PGD | PUD | PMD | PTE | 12-bit offset]
↓
PTE entry
↓
4 KiB frame

VA
[PGD | PUD | PMD | 21-bit offset]
↓
PMD entry with PS=1
↓
2 MiB physical frame

VA
[PGD | PUD | 30-bit offset]
↓
PUD entry with PS=1
↓
1 GiB physical frame

Huge page = early termination of walk
Use “page size” attribute in the entry
usually is 0 (meaning points to next level or
4KB page if leaf)
if set, then maps to 2MB or 1GB huge page

PTE in x86



Typical low-order flags:

bit 8	G	Global
bit 7	PAT	
bit 6	D	Dirty
bit 5	A	Accessed
bit 4	PCD	Cache disable
bit 3	PWT	Write-through
bit 2	U/S	User / Supervisor
bit 1	R/W	Read / Write
bit 0	P	Present

INVERTED PAGE TABLE

A single system-wide table mapping PFN to VPN + ASID

Naïve approach:

- Search through data structure $\langle \text{ppn}, \text{vpn} + \text{asid} \rangle$ to find match

- Too much time to search entire table

Better:

- Maintain a hash-map of $\text{vpn} + \text{asid}$ to ppn

- Smaller number of entries to search for exact match

Used in IBM PowerPC

- Typically requires TLB miss handling by software (OS)

SUMMARY: BETTER PAGE TABLES

Problem: Simple linear page tables require too much contiguous memory

Many options for efficiently organizing page tables

If OS traps on TLB miss, OS can use any data structure

- Eg. inverted page tables (hashing)

If Hardware handles TLB miss, page tables must follow specific format

- Multi-level page tables used in x86 architecture
- Each page table fits within a page

Next class: Swapping!